

DOCUMENTAÇÃO DO PROOF OF CONCEPT (POC)

1 REVISÃO BIBLIOGRÁFICA (MAPEAMENTO DE ARTIGOS)

Esta seção fundamenta teoricamente a aplicação de Modelos de Linguagem de Larga Escala (LLMs) na educação em computação, analisando estudos recentes que exploram a eficácia e os desafios dessa tecnologia.

ANÁLISE DE ESTUDOS RECENTES

Artigo 1: ScienceDirect (S2666827023000348)

Proposta: O estudo propõe uma revisão do modelo Performer (variante eficiente do Transformer) introduzindo uma operação FAVOR+ com cache, visando otimizar a geração de feedback textual para complexidade $O(L)$ em tempo e $O(1)$ em espaço. A arquitetura foi aplicada em um modelo de codificador/decodificador especializado para correção de código no ensino de Programação Orientada a Objetos (POO).

Conclusões: A revisão do modelo alcançou uma precisão METEOR superior a 96% e um aumento de mais de 120 vezes na velocidade de inferência em comparação a modelos tradicionais. Testes com usuários demonstraram que o feedback gerado foi mais benéfico para o aprendizado do que as respostas do ChatGPT, reforçando que modelos especializados podem superar generalistas em contextos educacionais específicos.

Artigo 2: ACM (10.1145/3587102.3588814)

Proposta: O estudo relata a experiência de utilizar o GPT-3 para resolver seis tarefas de um curso de Programação Orientada a Objetos (POO) em uma universidade portuguesa, utilizando enunciados escritos em português. A pesquisa buscou avaliar a capacidade do modelo em interpretar requisitos funcionais diretos e implementar múltiplas classes inter-relacionadas por composição ou herança seguindo boas práticas.

Conclusões: Os resultados indicam que o GPT-3 é capaz de interpretar requisitos funcionais e lidar com o idioma português sem dificuldade aparente, obtendo pontuações satisfatórias em ferramentas de avaliação automática baseadas em testes unitários. No entanto, a análise qualitativa revelou que o modelo tende a não fornecer as melhores soluções sob a perspectiva do design de software (POO), apresentando erros de lógica e dificuldades em seguir as melhores práticas arquiteturais.

Artigo 3: IEEE (10578838)

Proposta: O estudo investiga a aplicação do modelo GPT-4 para aprimorar o ensino de programação através da aplicação web "Tutor Kai". A ferramenta foi projetada para fornecer feedback pedagógico sobre tarefas de programação sem revelar a solução final, sendo avaliada por 51 estudantes durante um semestre letivo.

Conclusões: Os resultados demonstram que o GPT-4 identifica e menciona a maioria dos problemas presentes no código, tendo resolvido quase integralmente a limitação de expor a solução no feedback. Embora os alunos tenham avaliado a experiência positivamente (média 5,05/7), o estudo ressalta que a combinação de testes unitários falhos com soluções corretas pode induzir o modelo a identificar falhas inexistentes, evidenciando a necessidade de evitar o envio de informações contraditórias à IA.

Artigo 4: IEEE (11058691)

Proposta: O estudo realizou uma revisão sistemática da literatura analisando 32 trabalhos sobre a aplicação de LLMs no ensino de programação. A pesquisa examinou como essas tecnologias são aplicadas para automação de feedback, auxílio na depuração (correção de erros sintáticos e semânticos) e personalização da aprendizagem, além de investigar o impacto da Engenharia de Prompt na precisão das avaliações e os desafios de escalabilidade em cursos de larga escala.

Conclusões: Os autores concluíram que as LLMs são eficazes em aumentar a participação dos alunos e reduzir o tempo de conclusão de tarefas, sendo a Engenharia de Prompt um fator vital para a relevância do feedback. Contudo, o estudo alerta que os modelos ainda enfrentam dificuldades com problemas de raciocínio lógico de alto nível e que a dependência excessiva de feedbacks automatizados pode prejudicar o desenvolvimento do pensamento independente dos estudantes. Recomenda-se uma abordagem híbrida, que combine a automação com a supervisão humana para garantir a sustentabilidade pedagógica.

2 PASSO 1: VALIDAÇÃO DA CONECTIVIDADE E NÓ BÁSICO (PRÉ-LANGGRAPH)

Este passo inicial tinha como objetivo validar a conectividade com a API do Gemini, garantir a segurança das credenciais e refinar a Engenharia de Prompt para que a LLM gerasse a estrutura de saída rígida esperada para a orquestração futura.

2.1 CONECTIVIDADE E ROBUSTEZ DO CLIENTE

Objetivo: Estabelecer a comunicação funcional entre o ambiente Python e o modelo Gemini e testar a robustez contra falhas de rede.

2.1.1. Conclusões da Conectividade:

- 1. **Conectividade:** A comunicação com o modelo `gemini-2.5-flash` foi estabelecida com sucesso, usando a biblioteca oficial do Google GenAI.
- 2. **Segurança:** A utilização do arquivo `.env` para armazenar a `GEMINI_API_KEY` foi validada como a prática correta.
- 3. **Robustez (Tratamento de Erros):** A necessidade de implementar a lógica de *Exponential Backoff* foi confirmada em logs de erro. Esta função demonstrou ser crucial para a resiliência do sistema, permitindo que a aplicação se recuperasse automaticamente de limites de taxa temporários.

2.2 VALIDAÇÃO DA ESTRUTURA E DA ENGENHARIA DE PROMPT

Objetivo: Garantir que a saída da LLM obedeça à estrutura rígida (três pontos fixos) essencial para a leitura por máquina e a tomada de decisão pelo LangGraph.

2.2.1. Análise da Estrutura de Saída (Gatekeeper):

O prompt foi projetado para impor uma persona e, o mais importante, uma **Estrutura de Saída Rígida**, que atua como o *Gatekeeper* do LangGraph.

Tabela 1: Estrutura de Saída Rígida e Relação com o LangGraph

Estrutura de Saída Exigida	Por que é Crucial para o LangGraph
1. Avaliação:	O LangGraph usará este texto como Gatekeeper (Porteiro) para decidir o próximo nó.
2. Justificativa:	Fornece feedback qualitativo ao aluno.
3. Sugestão de Correção:	Ajuda o aluno a corrigir o código.

2.2.2. Resultados da Correção Simples (POC Inicial):

Os testes iniciais validaram que a LLM conseguia seguir a estrutura rígida e fornecer um feedback relevante.

Tabela 2: Resultados do POC - Validação Inicial do Prompt

Caso de Teste	Avaliação da LLM (Passo 1)	Conclusão
Código Correto	Avaliado como "Certo".	VALOR AGREGADO: Capacidade de ir além do enunciado, sugerindo boas práticas.
Código com Erro	Avaliado como "Parcialmente Certo"(na primeira tentativa).	VALIDAÇÃO ESTRUTURAL: O formato rígido foi respeitado, validando o Prompt.

3 PASSO 2: VALIDAÇÃO DA ORQUESTRAÇÃO COM LANGGRAPH

Este passo teve como objetivo testar a execução sequencial do nó de correção dentro de um grafo do LangGraph, garantindo que o sistema pudesse processar diferentes entradas de código de forma robusta e diferenciada.

3.1 RESULTADOS DA EXECUÇÃO SEQUENCIAL DOS CASOS DE TESTE

A execução de três casos de teste (Correto, Errado e Parcialmente Certo) em sequência foi concluída com sucesso, demonstrando a capacidade do LangGraph de gerenciar o estado e invocar a LLM repetidamente.

Tabela 3: Resultados da Execução Sequencial no LangGraph (Passo 2)

Caso de Teste (Código)	Avaliação da LLM	Conclusão da Análise
1. Código Correto	Certo	CONFIRMAÇÃO: A LLM manteve a capacidade de mentor, sugerindo boas práticas (construtor, validação de negativos).
2. Código com Erro	Errado	VALIDAÇÃO CRÍTICA: Confirma a detecção de falhas graves de POO (Encapsulamento e Lógica incorreta) e a obediência à Estrutura Rígida.
3. Código Parcialmente Certo	Parcialmente Certo	ROBUSTEZ: O teste foi concluído com sucesso, apesar de ter esbarrado no limite de taxa da API (recuperação via Exponential Backoff), e a avaliação identificou corretamente a falta de validação de valores de entrada.

4 PASSO 3: VALIDAÇÃO DA LEITURA E ESTUDO DE INPUT

Este passo tinha como objetivo principal validar a entrada de dados do mundo real por meio da leitura de arquivos e avaliar a melhor forma de enviar esse código para o modelo.

4.1 IMPLEMENTAÇÃO DA LEITURA DE ARQUIVOS

1. A leitura do enunciado do exercício (arquivo .txt) e do código do aluno (arquivo .java) foi implementada com sucesso no script `poc-leitura-arquivos-langgraph.py` utilizando a função nativa de I/O do Python.
2. O script `LangGraph` foi refatorado para receber o conteúdo lido dos arquivos como entrada no estado inicial, preparando o sistema para testes com o `*dataset*` completo.

4.2 AVALIAÇÃO DA ESTRATÉGIA DE ENVIO DE CÓDIGO ("ANEXO")

Foi avaliada a melhor estratégia para "anexar" o arquivo de código Java à LLM.

4.2.1. Conclusão da Estratégia:

A melhor prática para o envio de código-fonte foi confirmada como a leitura do conteúdo do arquivo para uma string ('str') e a inclusão dessa string dentro do prompt da LLM.

- **Eficiência:** Evita a sobrecarga de codificação (Base64) necessária para anexos binários (multimídia), que é desnecessária para código-fonte.
- **Confiança da LLM:** Enviar o código como texto formatado (usando `*code fences*` Markdown, `“java e “`) é o método mais confiável para garantir que o modelo interprete o conteúdo como código Java.
- **Simplicidade do Grafo:** Mantém a estrutura de estado ('CorrectionState') simples, utilizando apenas strings para tráfego de dados.

5 PASSO 4: CORREÇÃO DE PROJETO COM MÚLTIPLOS ARQUIVOS

O objetivo desta etapa era testar a capacidade da LLM de corrigir um projeto com múltiplos arquivos, exigindo correlação e análise de consistência entre classes.

5.1 ANÁLISE DO PROJETO MULTI-ARQUIVO

Projeto Testado: Sistema de Calculadora Simples.

5.1.1. Resultados e Validação:

A estratégia de concatenação dos arquivos (`ClassePrincipal.java` e `ClasseAuxiliar.java`) em uma única string, com delimitadores claros, demonstrou-se altamente eficaz:

- **Consistência Arquitetônica:** A LLM identificou o erro mais grave: a incompatibilidade entre a `ClassePrincipal` (que chamava o método `somar` de forma estática) e a `ClasseAuxiliar` (onde o método `somar` era de instância).
- **Erro Lógico:** O erro lógico (`return x - y;` em vez de `return x + y;`) dentro de `ClasseAuxiliar` foi corretamente detectado.
- **Feedback Integrado:** A sugestão de correção forneceu duas opções completas, corrigindo o código em **ambos os arquivos** de forma integrada e coerente, comprovando que o modelo tratou o input como um projeto único.
- **Conclusão do Passo 4:** O fluxo POC está validado para corrigir projetos Java multi-arquivo, superando a dificuldade de contexto de código distribuído.

6 PASSO 5: AVALIAÇÃO DE VIABILIDADE ECONÔMICA

O objetivo deste passo foi realizar uma análise de custo-benefício e viabilidade do projeto, com foco nas ferramentas de Inteligência Artificial Generativa.

6.1 CUSTOS DA API DO GEMINI

O modelo de preços da API do Gemini é baseado no pagamento por uso (*pay-as-you-go*), processado através do sistema *Cloud Billing*. A cobrança é realizada com base em vários fatores: Contagem de Tokens de Entrada (prompt e contexto), Contagem de Tokens de Saída (resposta gerada), e Tokens em Cache.

O `gemini-2.5-flash` oferece o melhor custo-benefício com o preço de entrada mais baixo, sendo ideal para tarefas de correção de alto volume, enquanto o `gemini-2.5-pro` oferece maior capacidade de raciocínio para projetos mais complexos a um custo mais elevado.

Solicitações de API que falham com erro 400 ou 500 não são cobradas. O uso do Google AI Studio para testes e desenvolvimento **permanece sem custo financeiro**, mesmo com o faturamento ativado. Créditos do Google Cloud são aceitos para o pagamento do uso da API Gemini.

6.2 LICENÇA ACADÊMICA E LIMITAÇÕES

Os limites de taxa são as métricas que regulam o número de solicitações que podem ser feitas à API Gemini em um determinado período. Esses limites são medidos em três dimensões principais e aplicados por projeto (não por chave de API):

- **Solicitações por Minuto (RPM):** O número máximo de requisições que podem ser feitas em 60 segundos.
- **Tokens por Minuto (TPM):** O volume máximo de tokens de entrada (prompt) que pode ser processado em 60 segundos.
- **Solicitações por Dia (RPD):** O número máximo de requisições que podem ser feitas em 24 horas (redefinidas à meia-noite do Horário do Pacífico). Esta é a principal restrição do Nível Grátis.

Exceder qualquer um desses limites (ex: 21 solicitações em um minuto com limite de RPM de 20) aciona um erro de limitação de taxa. Esses limites variam drasticamente dependendo do *status* do projeto (Grátis, Nível 1, Nível 2 ou Nível 3), sendo mais restritos para modelos experimentais e de prévia.

Dinâmica dos Limites de Taxa. Os limites de taxa da API Gemini dependem do nível de cota do projeto e são atualizados automaticamente à medida que o nível e o status da conta mudam. É crucial notar que os limites de taxa especificados não são garantidos, e a capacidade real pode variar dependendo da demanda do sistema e da região.

1. **Licença Acadêmica:** Não há uma "licença acadêmica" gratuita formalmente divulgada. O uso para pesquisa se enquadra no *Free Tier* ou no plano pago (*pay-as-you-go*). O plano pago é fundamental, pois garante que os dados do usuário e prompts **não serão utilizados para treinar os modelos do Google**.
2. **Limitações de Cota:** O *Free Tier* (**20 RPD**) mostrou-se impraticável para a POC (conforme logs do Passo 2). A solução para a escalabilidade é a ativação do faturamento no projeto do Google Cloud, removendo o limite de Requisições por Dia (RPD) e elevando as cotas de Requisições por Minuto (RPM) e Tokens por Minuto (TPM) para níveis que permitem uso intensivo.

6.3 ALTERNATIVAS DE LLMS

O mercado oferece diversas alternativas robustas para a correção de código, com variação notável em custo e capacidade de raciocínio. A tabela a seguir compara o custo por 1 milhão de tokens (1M T) dos principais concorrentes de baixo custo e alta performance, sendo a taxa de saída (Output) a métrica mais relevante para o custo final do feedback.

Tabela 4: Comparação de Custos por 1 Milhão de Tokens (1M T) - Modelos de Baixo Custo

Modelo	Custo Entrada (US\$/1M T)	Custo Saída (US\$/1M T)	Vantagem Competitiva
DeepSeek-V3.2	\$0.28	\$0.42	Mais baixo custo de saída (feedback), ideal para correção em massa e otimização de custo.
OpenAI GPT-5 mini	\$0.250	\$2.00	Mais baixo custo de entrada e alta velocidade (concorrente direto do Flash).
Google Gemini 2.5 Flash	\$0.30	\$2.50	Ótimo custo-benefício e janela de contexto de 1M (modelo usado na POC).

- **OpenAI (Modelos de Alta Performance):** GPT-5.2 Pro (Entrada \$21.00/1M T, Saída \$168.00/1M T) e Gemini 2.5 Pro (Entrada \$1.25/1M T, Saída \$10.00/1M T) são as opções de raciocínio mais avançado, sendo mais adequadas para problemas arquitetônicos complexos, mas com custo significativamente mais alto.
- **LLaMA/Mistral:** Implementação local (*self-hosted*) é a alternativa para custo zero de API, mas exige alto investimento inicial em infraestrutura de hardware (GPUs).

7 PASSO 6: ANÁLISE COMPARATIVA E CIENTÍFICA DO FEEDBACK

Este passo da POC analisou como a presença ou ausência de uma solução de referência (gabarito) altera a natureza da correção e a eficácia do feedback gerado pela LLM.

7.1 ESTUDO DE CASO I - EXERCÍCIO NAVE

7.1.1. COMPARAÇÃO FUNCIONAL POR ESTADO DE CORREÇÃO

A avaliação dos três cenários de teste revelou padrões distintos na forma como a IA processa as informações:

1. **Cenário Correto:** Com o gabarito, a IA valida a equivalência técnica e confirma a concordância a valores exatos previstos pelo professor. Sem o gabarito, a IA realiza uma va-

validação funcional, confirmando que os requisitos do enunciado foram atendidos de forma autônoma.

2. **Cenário Errado:** A presença do gabarito permite à IA identificar que o arquivo enviado é idêntico à versão inicial (não modificado). Mesmo sem acesso direto ao histórico de versões, a IA infere o estado inicial através da descrição do problema no enunciado e da ausência de alterações em relação ao código base. Na falta do gabarito a avaliação concentra-se na verificação dos conteúdos técnicos, mostrando a inexistência dos componentes solicitados nas instruções do exercício.
3. **Cenário Parcialmente Correto:** Com o suporte do gabarito, a IA realiza uma validação para identificar com precisão quais itens específicos do enunciado não foram satisfeitos. Sem o gabarito, a IA realiza uma análise lógica profunda, identificando inconsistências.

7.2 ESTUDO DE CASO II - EXERCÍCIO CASA

7.2.1. COMPARAÇÃO FUNCIONAL POR ESTADO DE CORREÇÃO (CASA)

Os testes realizados com o exercício “Casa” aprofundaram a compreensão sobre a sensibilidade da IA a variações paramétricas e controle de estado:

1. **Cenário Correto:** Com o suporte do gabarito, a IA demonstrou uma alta capacidade de discernimento ao classificar divergências numéricas (ex: distância de movimento de -310 vs -300) como triviais, mantendo o foco na integridade da funcionalidade. A análise validou com precisão a orquestração de eventos baseada em estados (booleano porDoSolRealizado). Sem o gabarito, o modelo manteve a eficácia na validação da sequência lógica, destacando o entendimento de fluxo e controle de estado como indicadores de proficiência do aluno.
2. **Cenário Errado:** Neste cenário, a presença do gabarito permite que a IA confronte o código com a arquitetura de referência da solução, detectando falhas de sintaxe, omissões em classes de suporte e o descumprimento de requisitos estruturais. Em contrapartida, sem o gabarito, além de apontar a ausência de requisitos funcionais, o modelo age como um analisador de compilação, explicando tecnicamente por que a estrutura escrita pelo aluno é inviável para execução.
3. **Cenário Parcialmente Correto:** Com a utilização do gabarito, a IA reconhece a criação e instanciação dos objetos, e identifica com precisão a omissão de métodos de modularização específicos (como o porDoSol1) exigidos na arquitetura. Na ausência do gabarito, a IA detecta que, apesar da existência das entidades na cena, a inexistência de uma lógica de controle impede o sequenciamento esperado, impossibilitando a interação entre o morador e o evento do pôr do sol.

7.2.2. RESUMO DA ANÁLISE CIENTÍFICA E CONCLUSÕES PARA O TCC

O experimento consolidado com os exercícios Nave e Casa comprovou três pilares fundamentais:

- **Precisão Técnica:** A IA demonstrou robustez pedagógica, validando corretamente conceitos avançados de POO, incluindo a gestão de estados de objetos e a interação entre múltiplas classes.
- **Contexto (IA como “Detetive”):** O gabarito atua como um ponto de referência. Ele permite que a IA diferencie entre erros conceituais e variações criativas ou menores.
- **Valor Pedagógico Superior:** Sem gabarito, a IA age como um analisador de compilação. Com gabarito, ela age como um analisador de arquitetura, confrontando o código do aluno com o design esperado pelo professor.